

Session 4

Strings, Number Checking, Conversion & DOM Fundamentals

JavaScript String Methods + Number Validation + Type Conversion + DOM Selection

SESSION 4 FOCUS

Learn how to inspect and transform strings, validate numeric data, convert values safely, and begin reading values from HTML elements with JavaScript.

Area	What you learn
String methods	Read, search, slice, replace, combine and inspect text
Number checking	isNaN and the Number.is* family
Number conversion	Number(), parseInt(), parseFloat() and when results differ
DOM basics	getElementById(), getElementsByTagName() and related selectors

FORMAT NOTE

This booklet uses a Mac/macOS-style clean text system with a modern sans-serif body face, dark editor-style code blocks, blue explanation cards and green result cards.

Table of Contents

Clickable index - select a topic to jump directly to its page

Page	Topic
04	Session 4 Roadmap
05	String Methods
06	length
07	toLowerCase() / toUpperCase()
08	split()
09	indexOf() / lastIndexOf()
10	slice()
11	match()
12	concat()
13	replace()
14	replaceAll()
15	startsWith() / endsWith()
16	charAt()
17	isWellFormed()
18	Related String Methods I
19	Related String Methods II
20	Related String Methods III
21	String Method Patterns + Mutation Note
22	Number Checking
23	isNaN()
24	Number.isNaN(), isFinite(), isInteger(), isSafeInteger()
25	Number Conversion

NAVIGATION

Every topic entry is clickable. Use the Return to Index button at the bottom of every page to come back here.

[Return to Index](#)

Table of Contents - Continued

Clickable index - continued

Page	Topic
26	Number()
27	parseInt()
28	parseFloat()
29	Number Conversion Comparison
30	DOM Basics - getElementById()
31	getElementsByTagName()
32	Related DOM Selectors
33	Collections + Duplicate ID Warning
34	getData() Walkthrough
35	HTML -> JavaScript Data Flow
36	Practice 1-10
37	Practice 11-20
38	Answer Key 1-10
39	Answer Key 11-20
40	Mini Project - Input Validator
41	Interview Revision
42	Quick Revision Sheet
43	Session 4 Summary
44	Clean Source Code - Part 1
45	Clean Source Code - Part 2
46	Clean Source Code - Part 3
47	Clean Source HTML

INDEX CONTINUATION

Second half of the Session 4 clickable index.

[Return to Index](#)

Session 4 Roadmap

How the concepts connect

A practical path from text processing to browser data access.

```
JavaScript Session 4
|
+-- String values
|   +-- inspect -> search -> extract -> replace -> test
|
+-- Number values
|   +-- check validity -> convert -> compare parsing strategies
|
+-- DOM
|   +-- find element -> read value -> work with collection -> use logic
```

CORE IDEA

Session 4 connects data handling with the browser. Strings are manipulated as values; numbers are checked and converted; DOM APIs let JavaScript read data from HTML.

String Methods

Overview of the session text toolkit

String methods let you inspect, search, transform and combine text. JavaScript string operations normally return a new value rather than changing the original string.

```
let str = "I am learning Javascript learning";  
|  
+-- Inspect: length, charAt, at  
+-- Search: indexOf, lastIndexOf, includes, search, match  
+-- Extract: slice, substring  
+-- Transform: toLowerCase, toUpperCase, replace, replaceAll  
+-- Combine: concat  
+-- Test: startsWith, endsWith, isWellFormed  
+-- Format: trim, repeat, padStart, padEnd
```

REMEMBER

Most String methods return a result. They do not rewrite the original primitive string in place. Store the result when you need the transformed value later.

length

String methods

Returns the number of UTF-16 code units in the string. In ordinary text this behaves like a character count. Spaces are counted.

```
let n = str.length;
```

WHAT IT DOES

Use `.length` when you need the size of a string or array. It is a property, not a function - do not write `str.length()`.

```
let str = "I am learning Javascript learning";  
console.log(str.length);
```

EXPECTED RESULT

30

toLowerCase() / toUpperCase()

String methods

Creates lowercase or uppercase versions of the string.

```
str.toLowerCase();  
str.toUpperCase();
```

WHAT IT DOES

Both methods return a new string value; the original `str` remains unchanged.

```
console.log(str.toLowerCase());  
console.log(str.toUpperCase());
```

EXPECTED RESULT

i am learning javascript learning I AM LEARNING JAVASCRIPT LEARNING

COMMON PITFALL

Locale-specific casing rules can be more complex than simple ASCII text; these methods are suitable for the basic English examples used here.

split()

String methods

Splits a string into an array. With no separator, the whole string becomes one array element.

```
str.split(separator);
```

WHAT IT DOES

The separator controls how the string is divided. `split(" ")` creates words; `split("")` creates UTF-16 code units, which is not always the same as user-perceived characters.

```
console.log(str.split());  
console.log(str.split(" "));
```

EXPECTED RESULT

```
["I am learning Javascript learning"] ["I", "am", "learning", "Javascript", "learning"]
```

indexOf() / lastIndexOf()

String methods

Finds the first or last occurrence of a search value and returns its zero-based index. If nothing is found, the result is `-1`.

```
str.indexOf(searchValue);  
str.lastIndexOf(searchValue);
```

WHAT IT DOES

Do not use a truthy check like `if (str.indexOf("x"))` because index `0` is a valid match but is falsy. Compare with `!== -1` when checking existence.

```
console.log(str.indexOf("r"));  
console.log(str.lastIndexOf("m"));
```

EXPECTED RESULT

8 3

COMMON PITFALL

Case matters: `indexOf("R")` and `indexOf("r")` are different searches.

slice()

String methods

Extracts part of a string from `start` up to, but not including, `end`. Negative indexes count from the end.

```
str.slice(start, end);
```

WHAT IT DOES

The end index is excluded. `slice(-1)` returns the final character.

```
console.log(str.slice(-1));  
console.log(str.slice(5, 12));
```

EXPECTED RESULT

g learning

COMMON PITFALL

Because the end position is exclusive, `slice(0, 3)` returns indexes 0, 1 and 2.

match()

String methods

Searches using a regular expression and returns matched text. With the global ``g`` flag, all matches are returned.

```
str.match(pattern);
```

WHAT IT DOES

``match()`` is useful when searching with patterns rather than a simple literal string. When there is no match, the result can be ``null``.

```
console.log(str.match(/learning/g));
```

EXPECTED RESULT

```
["learning", "learning"]
```

COMMON PITFALL

The regular expression flag ``g`` means global matching.

concat()

String methods

Joins additional strings to the end of the original string and returns the combined string.

```
str.concat(value1, value2, ...);
```

WHAT IT DOES

For simple concatenation, the `+` operator is also common: `str + ", Great"`. `concat()` is a direct string method.

```
console.log(str.concat(", Great"));
```

EXPECTED RESULT

I am learning Javascript learning, Great

replace()

String methods

Replaces the first occurrence of the search value.

```
str.replace(searchValue, newValue);
```

WHAT IT DOES

With a plain string search, only the first matching occurrence is replaced.

```
console.log(str.replace("learning", "hey"));
```

EXPECTED RESULT

I am hey Javascript learning

COMMON PITFALL

Use `replaceAll()` or a global regular expression when every occurrence must be replaced.

replaceAll()

String methods

Replaces every occurrence of the search value.

```
str.replaceAll(searchValue, newValue);
```

WHAT IT DOES

Use `replaceAll()` when you want a direct all-occurrences replacement.

```
console.log(str.replaceAll("learning", "hey"));
```

EXPECTED RESULT

I am hey Javascript hey

COMMON PITFALL

Do not confuse `replace()` with `replaceAll()` when repeated text is possible.

startsWith() / endsWith()

String methods

Tests whether a string begins or ends with a specified sequence. Both return Boolean values.

```
str.startsWith(search);  
str.endsWith(search);
```

WHAT IT DOES

These methods are ideal for clear prefix/suffix checks.

```
console.log(str.startsWith("I"));  
console.log(str.endsWith("g"));
```

EXPECTED RESULT

true true

COMMON PITFALL

They are case-sensitive and return only `true` or `false`.

charAt()

String methods

Returns the character at a specified zero-based index.

```
str.charAt(index);
```

WHAT IT DOES

For modern code, `str[index]` and `str.at(index)` are also common ways to access characters.

```
console.log(str.charAt(0));  
console.log(str.charAt(5));
```

EXPECTED RESULT

l m

COMMON PITFALL

An out-of-range `charAt()` call returns an empty string.

isWellFormed()

String methods

Checks whether the string contains well-formed Unicode code units. Normal text returns `true`.

```
str.isWellFormed();
```

WHAT IT DOES

This method is useful when code may receive arbitrary Unicode text and you need to detect lone surrogate code units.

```
console.log(str.isWellFormed());
```

EXPECTED RESULT

true

COMMON PITFALL

It is more specialized than the other beginner methods, but it is included because it appears in the session source.

Related String Methods I

Useful methods not present in the original source

The source session already covered the core methods. The following related methods complete the same conceptual family.

Method	Purpose	Example
includes()	Checks whether a string contains a sequence.	<code>str.includes("learning")</code>
search()	Returns the first regex match index, or -1.	<code>str.search(/learning/)</code>
substring()	Extracts from one index to another; negative values are treated as 0.	<code>str.substring(0, 2)</code>

```
console.log(str.includes("learning"));  
console.log(str.search(/learning/));  
console.log(str.substring(0, 4));
```

WHEN TO USE WHICH

Use `includes()` for a simple yes/no containment check, `search()` when you want a regular-expression search index, and `substring()` when you need a non-negative index-based extraction rule.

Related String Methods II

Whitespace, indexing and character inspection

Method	Purpose	Example
trim()	Removes whitespace from both ends.	input.trim()
trimStart()	Removes leading whitespace.	input.trimStart()
trimEnd()	Removes trailing whitespace.	input.trimEnd()
at()	Reads a character by index and supports negative indexes.	str.at(-1)
charCodeAt()	Returns the UTF-16 code unit at an index.	str.charCodeAt(0)

```
let input = "  Hello  ";
console.log(input.trim());
console.log(str.at(-1));
console.log(str.charCodeAt(0));
```

IMPORTANT

`at(-1)` is a convenient modern way to read the final character. `charCodeAt()` returns a numeric UTF-16 code unit, so it is lower-level than `charAt()`.

Related String Methods III

Formatting and repeated-pattern helpers

Method	Purpose	Example
repeat()	Repeats a string a chosen number of times.	"-".repeat(5)
padStart()	Pads the beginning until a target length is reached.	"7".padStart(3,"0")
padEnd()	Pads the end until a target length is reached.	"7".padEnd(3,"0")
matchAll()	Returns an iterator over all regex matches with capture groups.	str.matchAll(/learning/g)

```
console.log("-".repeat(5));  
console.log("7".padStart(3, "0"));  
console.log("7".padEnd(3, "0"));
```

PRACTICAL USE

These helpers are useful for formatting, report output and controlled text patterns. `matchAll()` becomes particularly useful when capture groups are needed from multiple regex matches.

String Method Patterns + Mutation Note

How to think about String APIs

A string is a primitive value. Methods such as `toUpperCase()`, `replace()` and `trim()` return another string value instead of mutating the original primitive.

```
let text = " hello ";
let cleaned = text.trim();
let upper = cleaned.toUpperCase();
console.log(text);    // original value
console.log(upper);   // transformed result
```

PATTERN

Think in terms of: original value -> method call -> returned value. Save the returned value in a variable whenever the transformed result is needed later.

Number Checking

Validation family overview

JavaScript provides the global `isNaN()` function and a `Number.is*` family for more explicit numeric checks.

Tool	Main question
<code>isNaN(value)</code>	Does conversion of value result in NaN?
<code>Number.isNaN(value)</code>	Is value actually the NaN value?
<code>Number.isFinite(value)</code>	Is value a finite number?
<code>Number.isInteger(value)</code>	Is value an integer?
<code>Number.isSafeInteger(value)</code>	Is value a safe integer?

WHY THIS MATTERS

Validation should answer a precise question. For modern code, the `Number.is*` methods are often easier to reason about because they do not perform the broad coercion behavior of the global `isNaN()`.

isNaN()

Global numeric validity check

The global `isNaN()` function checks whether a value becomes NaN after numeric conversion.

```
let a = 4;
console.log(isNaN(a));
if (isNaN(a)) {
  console.log("Enter a valid number");
} else {
  console.log("Valid number");
}
```

EXPECTED RESULT

false Valid number

WARNING

Because the global function converts its argument first, it can produce results that are surprising for strings such as `"123"`.

Number.isNaN(), isFinite(), isInteger(), isSafeInteger()

More explicit number checks

Method	Example	Result meaning
Number.isNaN(x)	Number.isNaN(NaN)	true only for NaN
Number.isFinite(x)	Number.isFinite(12)	true for finite numbers
Number.isInteger(x)	Number.isInteger(12)	true for integers
Number.isSafeInteger(x)	Number.isSafeInteger(12)	true within safe integer range

```
console.log(Number.isNaN(NaN));  
console.log(Number.isFinite(12));  
console.log(Number.isInteger(12));  
console.log(Number.isSafeInteger(12));
```

RULE OF THUMB

Prefer the `Number.is\*` methods when you want an explicit check against a numeric condition without broad type coercion.

Number Conversion

Why `Number()` and parsing are different

The source demonstrates a string containing digits followed by letters: `"12.34abc"`. Different conversion functions interpret that value differently.

```
let num1 = "12.34abc";  
Number(num1);      // NaN  
parseInt(num1);     // 12  
parseFloat(num1);   // 12.34
```

CONCEPT

`Number()` expects the whole string to represent a valid number. `parseInt()` and `parseFloat()` read a numeric prefix and stop when parsing cannot continue.

Number()

Whole-value conversion

`Number(value)` attempts to convert the entire value to a JavaScript number.

```
console.log(Number("12.34"));  
console.log(Number("12.34abc"));  
console.log(Number(""));
```

OUTPUT

12.34 NaN 0

TIP

For user input that should represent an entire numeric value, `Number()` is a straightforward choice. Validate the result when input may be invalid.

parseInt()

Parse an integer prefix

`parseInt(value, radix)` reads an integer from the beginning of a string. The radix argument is recommended when interpreting non-decimal strings.

```
console.log(parseInt("12.34abc", 10));  
console.log(parseInt("101", 2));  
console.log(parseInt("42px", 10));
```

OUTPUT

12 5 42

IMPORTANT

`parseInt()` stops at the first character that cannot be part of the parsed integer. It is a parsing function, not a strict full-string validator.

parseFloat()

Parse a decimal prefix

`parseFloat()` reads a floating-point number from the beginning of a string.

```
console.log(parseFloat("12.34abc"));  
console.log(parseFloat("42px"));  
console.log(parseFloat("3.14"));
```

OUTPUT

12.34 42 3.14

PAIR WITH parseInt()

Use `parseInt()` when you want an integer prefix; use `parseFloat()` when a decimal prefix is expected.

Number Conversion Comparison

One string, three outcomes

Input	Number()	parseInt()	parseFloat()
"12.34abc"	NaN	12	12.34
"42px"	NaN	42	42
"12.34"	12.34	12	12.34
""	0	NaN	NaN

MEMORIZE THE PATTERN

Number() = strict whole-value conversion; parseInt() = integer prefix; parseFloat() = decimal prefix.

DOM Basics - getElementById()

Read an element by its unique id

The DOM represents the HTML document as objects that JavaScript can access. `document.getElementById()` returns the element whose `id` matches the given string.

```
const res = document.getElementById("name");  
console.log(res.value);
```

SOURCE CONNECTION

The session HTML contains an input element with `id="name"`, and the JavaScript reads its `.value` inside `getData()`.

IMPORTANT

An `id` should identify one element in a document. If the same id appears on multiple elements, id-based selection becomes invalid or misleading.

getElementsByTagName()

Collect elements by tag name

`document.getElementsByTagName("p")` returns a collection of all matching paragraph elements.

```
const paragraphs = document.getElementsByTagName("p");  
console.log(paragraphs);  
console.log(paragraphs[0].textContent);
```

SOURCE CONNECTION

The supplied code uses `document.getElementsByTagName("p")` and logs the returned collection.

COLLECTION NOTE

The result is an `HTMLCollection`. It can contain zero, one or many matching elements. Access an individual item by index when needed.

Related DOM Selectors

Useful selectors not present in the source

Method	Purpose	Example
querySelector()	Returns the first element matching a CSS selector.	<code>document.querySelector("p")</code>
querySelectorAll()	Returns all matching elements as a <code>NodeList</code> .	<code>document.querySelectorAll("p")</code>
getElementsByClassName()	Returns elements with a given class name.	<code>document.getElementsByClassName("item")</code>

WHY ADD THESE

They are natural next steps from `getElementById()` and `getElementsByTagName()`. They let you select elements by CSS-style selectors or class names.

Collections + Duplicate ID Warning

Important DOM behavior

```
<div id="para1">1</div>
<div id="para1">2</div>
<p>Lorem ipsum dolor sit.</p>
```

PROBLEM IN THE SOURCE HTML

The provided HTML declares `id="para1"` twice. IDs are intended to be unique. Replace repeated ids with classes or unique ids when multiple elements share a role.

COLLECTION THINKING

Tag/class selectors may return collections. An id selector is intended to return one specific element.

getData() Walkthrough

Read input + inspect paragraph elements

```
function getData() {  
  const res = document.getElementById("name").value;  
  console.log("res", res);  
  
  const para1 = document.getElementsByTagName("p");  
  console.log("para1 => ", para1);  
}
```

Step	What happens
1	User enters text in the input.
2	getData() is triggered by the button click.
3	getElementById("name") finds the input element.
4	.value reads the current input text.
5	getElementsByTagName("p") retrieves paragraph elements.

SOURCE HTML CONNECTION

The button uses `onclick="getData()"`, so the JavaScript function runs when the user clicks Enter.

HTML -> JavaScript Data Flow

The browser interaction model

```
HTML input
|
+-- id="name"
|
button onclick="getData()"
|
v
getData()
|
+-- document.getElementById("name")
|   +-- .value
|
+-- document.getElementsByTagName("p")
|   +-- HTMLCollection
```

KEY IDEA

The DOM is the bridge between the HTML structure and JavaScript logic. Select an element first; then read a property such as `.value` or `.textContent`.

Practice 1-10

Session 4 exercises

Solve these without looking at the answer key first.

#	Problem
1	Print the length of a user-provided string.
2	Convert a string to lowercase and uppercase.
3	Count whether the word "learning" exists using includes().
4	Find the first and last index of a chosen character.
5	Extract the last 3 characters using slice().
6	Replace the first occurrence of a word.
7	Replace all occurrences of a word.
8	Check whether a string starts with a chosen prefix.
9	Check whether a string ends with a chosen suffix.
10	Use charAt() and at() to read characters from a string.

Practice 11-20

Number, conversion and DOM exercises

Continue with mixed Session 4 problems.

#	Problem
11	Check whether a value is NaN using isNaN().
12	Compare global isNaN() with Number.isNaN().
13	Check whether a number is finite and an integer.
14	Convert "123.45abc" using Number(), parseInt() and parseFloat().
15	Ask for a number and reject invalid input.
16	Read an input element by id and print its value.
17	Read all paragraph elements and print the first paragraph text.
18	Use querySelector() to select the first paragraph.
19	Use querySelectorAll() to count matching paragraphs.
20	Fix the duplicate id issue in the supplied HTML.

Answer Key 1-10

Suggested solutions

Use these as reference solutions after attempting the questions yourself.

```
1. console.log(value.length);
2. console.log(value.toLowerCase(), value.toUpperCase());
3. console.log(value.includes("learning"));
4. console.log(value.indexOf(target), value.lastIndexOf(target));
5. console.log(value.slice(-3));
6. console.log(value.replace(target, replacement));
7. console.log(value.replaceAll(target, replacement));
8. console.log(value.startsWith(prefix));
9. console.log(value.endsWith(suffix));
10. console.log(value.charAt(0), value.at(-1));
```

LEARNING POINT

These solutions deliberately use the same method families taught in Session 4. Try changing the test values to make sure you understand the result.

Answer Key 11-20

Suggested solutions

```
11. console.log(isNaN(value));
12. console.log(isNaN("123x"), Number.isNaN("123x"));
13. console.log(Number.isFinite(value), Number.isInteger(value));
14. console.log(Number(s), parseInt(s, 10), parseFloat(s));
15. const n = Number(input); console.log(Number.isNaN(n) ? "Invalid" : "Valid");
16. console.log(document.getElementById("name").value);
17. console.log(document.getElementsByTagName("p")[0].textContent);
18. console.log(document.querySelector("p"));
19. console.log(document.querySelectorAll("p").length);
20. Replace duplicate ids with unique ids or a shared class.
```

DOM REMINDER

Selection and reading are separate steps: find the element first, then read a property such as `.value`` or `.textContent``.

Mini Project - Input Validator

Combine the Session 4 concepts

Build a small browser program with one text input and one button.

```
User enters text
|
v
trim() -> empty check
|
+-- if empty -> show message
|
+-- else -> display uppercase text
      + length
      + startsWith / endsWith test
      + Number() validity check when numeric
```

Requirement	Concept
Read input	getElementById() + .value
Clean input	trim()
Text analysis	length, includes(), startsWith(), endsWith()
Text transform	toUpperCase() / replaceAll()
Numeric branch	Number() + Number.isNaN()
Output	console.log() or DOM text update

Interview Revision

Fast answers for Session 4

Question	Answer
What does length return?	The string length as UTF-16 code units.
indexOf() vs lastIndexOf()?	First match vs last match index.
slice() end index?	The end position is exclusive.
replace() vs replaceAll()?	First replacement vs all occurrences.
What does isNaN() do?	Checks whether a value becomes NaN after numeric conversion.
Number.isNaN()?	Checks whether the value itself is NaN without broad coercion.
Number() vs parseInt()?	Whole-value conversion vs integer-prefix parsing.
getElementById()?	Selects the element with a matching id.
getElementsByName()?	Returns matching elements as an HTMLCollection.
Why avoid duplicate ids?	An id is intended to be unique in the document.

INTERVIEW TIP

Give a one-sentence definition first, then a one-line example. Clear terminology matters more than memorizing long descriptions.

Quick Revision Sheet

Session 4 at a glance

```
STRING
length | toLowerCase | toUpperCase | split
indexOf | lastIndexOf | slice | match
concat | replace | replaceAll
startsWith | endsWith | charAt | isWellFormed
includes | search | substring | trim | at
repeat | padStart | padEnd | matchAll

NUMBER CHECKING
isNaN | Number.isNaN | Number.isFinite
Number.isInteger | Number.isSafeInteger

NUMBER CONVERSION
Number() | parseInt() | parseFloat()

DOM
getElementById | getElementsByTagName
querySelector | querySelectorAll | getElementsByClassName
```

MEMORY MAP

String -> inspect/search/transform. Numbers -> validate/convert. DOM -> select/read.

Session 4 Summary

What you should be able to do now

After Session 4, you should be able to:

Skill	Expected ability
String inspection	Measure length and read characters
String searching	Find text using index, includes or regex-based methods
String extraction	Use slice and substring
String transformation	Use case conversion and replacement methods
String formatting	Trim, repeat and pad text
Number validation	Use isNaN and Number.is* checks
Number conversion	Choose Number, parseInt or parseFloat for the task
DOM selection	Read elements by id, tag name and CSS selectors
DOM collections	Work with HTMLCollection / NodeList results
Browser input	Connect an HTML input and button to a JavaScript function

NEXT STEP

Use these methods inside conditions, loops and small DOM-driven applications. Session 4 gives you the data-handling tools needed before larger browser projects.

Clean Source Code - Part 1

Appendix: original Session 4 JavaScript

```
// String Methods

let str = "I am learning Javascript learning";
// Creates a variable named 'str' and stores the string in it.
// String value: "I am learning Javascript learning"

console.log(str.length);
// .length returns the total number of characters in the string.
// Spaces are also counted.
// Output: 30

console.log(str.toLowerCase());
// Converts the entire string to lowercase.
// Output: "i am learning javascript learning"

console.log(str.toUpperCase());
// Converts the entire string to uppercase.
// Output: "I AM LEARNING JAVASCRIPT LEARNING"

console.log(str.split());
// Splits the string into an array.
// Without an argument, the entire string becomes one array element.
// Output: ["I am learning Javascript learning"]

console.log(str.indexOf("r"));
// Finds the position (index) of the first occurrence of "r".
// JavaScript uses zero-based indexing.
// Output: 8

console.log(str.lastIndexOf("m"));
// Finds the position of the last occurrence of "m".
// Output: 3

console.log(str.slice(-1));
// Extracts characters from the given position to the end.
// -1 means the last character.
// Output: "g"

console.log(str.match(/learning/g));
// Finds all occurrences of "learning" using a regular expression.
// The 'g' flag means "global", so all matches are returned.
// Output: ["learning", "learning"]

console.log(str.concat(", Great"));
// Joins another string to the end of the original string.
// Output: "I am learning Javascript learning, Great"

console.log(str.replace("learning", "hey"));
// Replaces only the FIRST occurrence of "learning".
// Output: "I am hey Javascript learning"

console.log(str.replaceAll("learning", "hey"));
// Replaces ALL occurrences of "learning".
// Output: "I am hey Javascript hey"
```

Clean Source Code - Part 2

Appendix: original Session 4 JavaScript

```
console.log(str.startsWith("I"));
// Checks whether the string starts with "I".
// Returns true or false.
// Output: true

console.log(str.endsWith("g"));
// Checks whether the string ends with "g".
// Returns true or false.
// Output: true

console.log(str.charAt(0));
// Returns the character at index 0.
// Index 0 is the first character.
// Output: "I"

console.log(str.isWellFormed());
// Checks whether the string contains well-formed Unicode characters.
// Normal text like this returns true.
// Output: true

// Number Checking

let a = 4;
// Creates a variable 'a' and assigns the number 4 to it.

console.log(isNaN(a));
// isNaN() checks whether a value is "Not a Number".
// Since 4 is a valid number, the result is false.
// Output: false

if (isNaN(a)) {
    // If 'a' is Not a Number, this block will execute.

    console.log("Enter a valid number");
    // Displays a message asking the user to enter a valid number.
} else {
    // If 'a' is a valid number, this block will execute.

    console.log("Valid number");
    // Displays "Valid number".
}

// Number Conversion

let num1 = "12.34abc";
// Stores the text "12.34abc" in the variable num1.
// Notice that this is a STRING, not a number.

console.log(Number(num1));
// Number() tries to convert the entire string into a number.
// "12.34abc" contains non-numeric characters, so conversion fails.
// Output: NaN
```

Clean Source Code - Part 3

Appendix: original Session 4 JavaScript

```
console.log(parseInt(num1));  
// parseInt() reads the number from the beginning of the string.  
// It stops when it encounters a character that is not part of the integer.  
// Output: 12  
  
function getData() {  
  const res = document.getElementById("name").value  
  console.log('res', res)  
  
  const para1 = document.getElementsByTagName('p')  
  console.log("para1 => ", para1)  
  
}
```

Clean Source HTML

Appendix: original Session 4 HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="s4.js"></script>
</head>

<body>
  <input type="text" id="name">
  <button onclick="getData()">Enter</button>
  <div id="para1">1</div>
  <div id="para1">2</div>
  <p>Lorem ipsum dolor sit.</p>
</body>

</html>
```

HTML QUALITY NOTE

The source contains duplicate `id="para1"` values. Keep ids unique when connecting HTML elements to JavaScript.